

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА**

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №1

з дисципліни **“Функціональне програмування”**

на тему **“ОСНОВИ ФУНКЦІОНАЛЬНОГО ПРОГРАМУВАННЯ У
PYTHON”**

Варіант 1. Обробка замовлень інтернет-магазину

Виконав:

Студент групи ФЕП-23

Маньківський Максим Володимирович

Перевірив:

проф. Свелеба С. А.

Львів-2026

Мета: зрозуміти поняття чистої функції та референтної прозорості, навчитися виявляти й ізолювати побічні ефекти, переписати імперативний фрагмент обробки замовлень у функціональному стилі та параметризувати обчислення за допомогою `typing.Callable`.

Теоретичні відомості:

Чиста функція — це функція, яка детермінована (однаковий вхід завжди дає однаковий вихід), не змінює глобального стану та вхідних даних і не виконує введення-виведення. Вираз, побудований із чистих функцій, є референтно прозорим: його можна замінити на обчислене значення без зміни поведінки програми, що спрощує тестування, мемоізацію та міркування про код.

Побічними ефектами є I/O (`print`, файли, мережа), мутація вхідних структур, читання або зміна глобальних змінних, залежність від часу, випадковості чи оточення. Функціональний підхід полягає у відокремленні чистого ядра, яке лише перетворює дані, від тонкої оболонки ефектів, яка читає вхід і виводить результат. Залежності на кшталт годинника або генератора випадкових чисел передаються в ядро як функції (інжекція залежностей), а правила обчислень (знижка, податок, фільтр) — як значення типу `Callable`, що робить код компонувальним: будь-яку політику можна замінити, передавши іншу функцію.

Хід роботи:

1. Проаналізував стартовий імперативний фрагмент `starter_imperative.py`. У ньому є чотири проблеми: глобальна константа `TAX_RATE`, виклик `print`

усередині обчислень, мутація вхідних словників (o["total"] = ...) та змішування обчислення з I/O:

```
5 TAX_RATE = 0.2
6
7
8 def process_orders(orders: list[Any], min_total: float, discount: float) -> dict[str, Any]:
9     valid: list[dict[str, Any]] = []
10    total_revenue = 0.0
11
12    for o in orders:
13        if not o["paid"]:
14            continue
15        print("Processing order:", o["id"])
16
17        order_total = 0.0
18        for it in o["items"]:
19            order_total += it["price"] * it["qty"]
20        if order_total < min_total:
21            continue
22
23        order_total = order_total * (1 - discount)
24        order_total = order_total * (1 + TAX_RATE)
25
26        o["total"] = order_total
27        valid.append(o)
28        total_revenue += order_total
29
30    return {"count": len(valid), "revenue": total_revenue, "orders": valid}
```

2. У файлі core.py описав типи даних через TypedDict (Item, Order, Report), псевдоніми політик на основі Callable та дві базові чисті функції: order_subtotal обчислює суму позицій, а with_total повертає новий словник, не змінюючи вхідний:

```
9 from collections.abc import Callable, Iterable
10 from typing import TypedDict, TypeVar
11
12
13 class Item(TypedDict):
14     price: float
15     qty: int
16
17
18 class Order(TypedDict, total=False):
19     id: int
20     items: list[Item]
21     paid: bool
22     total: float
23
24
25 class Report(TypedDict):
26     count: int
27     revenue: float
28     orders: list[Order]
29
30
31 FilterFn = Callable[[float], bool]
32 DiscountFn = Callable[[float], float]
33 TaxFn = Callable[[float], float]
34 Processor = Callable[[Iterable[Order]], Report]
35 NowFn = Callable[[], float]
36
37 A = TypeVar("A")
38 B = TypeVar("B")
39 C = TypeVar("C")
40
41
42 def order_subtotal(order: Order) -> float:
43     return sum(it["price"] * it["qty"] for it in order["items"])
44
45
46 def with_total(order: Order, total: float) -> Order:
47     return {**order, "total": total}
```

3. Реалізував фабрику `make_processor`, яка приймає три політики (асерт, `apply_discount`, `apply_tax`) і повертає налаштований обробник. Обробник відбирає оплачені замовлення через `list comprehension`, застосовує композицію знижки та податку і повертає новий результат. Функція `process_orders_pure` є окремим випадком: вона збирає обробник із фабрик політик `min_total_filter`, `percent_discount` і `flat_tax`, отримуючи всі параметри явно:

```
50 def make_processor(  
51     *,  
52     accept: FilterFn,  
53     apply_discount: DiscountFn,  
54     apply_tax: TaxFn,  
55 ) -> Processor:  
56     def process(orders: Iterable[Order]) -> Report:  
57         qualified = [  
58             with_total(o, apply_tax(apply_discount(order_subtotal(o))))  
59             for o in orders  
60             if o["paid"] and accept(order_subtotal(o))  
61         ]  
62         return {  
63             "count": len(qualified),  
64             "revenue": sum(o["total"] for o in qualified),  
65             "orders": qualified,  
66         }  
67     return process  
68  
69  
70  
71 def min_total_filter(min_total: float) -> FilterFn:  
72     return lambda subtotal: subtotal >= min_total  
73  
74  
75 def percent_discount(rate: float) -> DiscountFn:  
76     return lambda amount: amount * (1 - rate)  
77  
78  
79 def flat_tax(rate: float) -> TaxFn:  
80     return lambda amount: amount * (1 + rate)  
81  
82  
83 def process_orders_pure(  
84     orders: Iterable[Order],  
85     *,  
86     min_total: float,  
87     discount: float,  
88     tax_rate: float,  
89 ) -> Report:  
90     process = make_processor(  
91         accept=min_total_filter(min_total),  
92         apply_discount=percent_discount(discount),  
93         apply_tax=flat_tax(tax_rate),  
94     )  
95     return process(orders)
```

4. Додав функції з додаткових вправ: узагальнену композицію `compose`, фабрику множників `make_multiplier` та `stamp_total`, яка отримує годинник ззовні як `Callable`, тому залишається тестованою без залежності від реального часу:

```
98 def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:  
99     return lambda x: f(g(x))  
100  
101  
102 def make_multiplier(k: int) -> Callable[[int], int]:  
103     return lambda x: x * k  
104  
105  
106 def stamp_total(total: float, now: NowFn) -> tuple[float, float]:  
107     return total, now()
```

5. Усі побічні ефекти виніс в оболонку `app.py`: читання замовлень із JSON-файлу, функцію `render_report`, що лише перетворює чистий результат на текст, друк та передачу `time.time` у ядро:

```
10 from core import Order, Report, process_orders_pure, stamp_total
11
12 SAMPLE: list[Order] = [
13     {"id": 1, "items": [{"price": 50.0, "qty": 3}], "paid": True},
14     {"id": 2, "items": [{"price": 20.0, "qty": 1}], "paid": True},
15     {"id": 3, "items": [{"price": 200.0, "qty": 1}], "paid": False},
16     {"id": 4, "items": [{"price": 10.0, "qty": 10}, {"price": 5.5, "qty": 2}], "paid": True},
17 ]
18
19
20 def load_orders(path: Path) -> list[Order]:
21     return list(json.loads(path.read_text(encoding="utf-8")))
22
23
24 def render_report(result: Report) -> str:
25     lines = [f"Processed: {o['id']} total: {o['total']:.2f}" for o in result["orders"]]
26     return "\n".join(*lines, f"Count: {result['count']}", f"Revenue: {result['revenue']:.2f}")
27
28
29 def main(argv: list[str]) -> None:
30     orders = load_orders(Path(argv[1])) if len(argv) > 1 else SAMPLE
31     result = process_orders_pure(orders, min_total=100, discount=0.1, tax_rate=0.2)
32     print(render_report(result))
33     revenue, stamp = stamp_total(result["revenue"], time.time)
34     print(f"Stamped: {revenue:.2f} at {time.strftime('%H:%M:%S', time.localtime(stamp))}")
35
36
37 if __name__ == "__main__":
38     main(sys.argv)
```

6. Написав `pytest`-тести у `tests/test_core.py`: референтна прозорість, відсутність мутації вхідних даних, еквівалентність імперативній версії та правильність фільтрації:

```
21 ARGS = {"min_total": 100.0, "discount": 0.1, "tax_rate": 0.2}
22
23
24 def test_referential_transparency(sample_orders: list[Order]) -> None:
25     r1 = process_orders_pure(sample_orders, **ARGS)
26     r2 = process_orders_pure(sample_orders, **ARGS)
27     assert r1 == r2
28
29
30 def test_no_mutation(sample_orders: list[Order]) -> None:
31     original = deepcopy(sample_orders)
32     process_orders_pure(sample_orders, min_total=0, discount=0.0, tax_rate=0.0)
33     assert sample_orders == original
34
35
36 def test_matches_imperative(sample_orders: list[Order]) -> None:
37     expected = process_orders(deepcopy(sample_orders), 100, 0.1)
38     assert process_orders_pure(sample_orders, **ARGS) == expected
39
40
41 def test_filters_unpaid_and_small(sample_orders: list[Order]) -> None:
42     result = process_orders_pure(sample_orders, **ARGS)
43     assert [o["id"] for o in result["orders"]] == [1, 4]
44     assert result["count"] == 2
45     assert result["revenue"] == pytest.approx(150 * 0.9 * 1.2 + 111 * 0.9 * 1.2)
46
47
48 def test_order_subtotal() -> None:
49     assert order_subtotal({"id": 1, "items": [{"price": 2.5, "qty": 4}], "paid": True}) == 10.0
50     assert order_subtotal({"id": 2, "items": [], "paid": True}) == 0.0
51
52
53 def test_with_total_returns_new_dict() -> None:
54     order: Order = {"id": 1, "items": [], "paid": True}
```

7. Додав тести на Callable-політики (make_processor з лямбдами), фабрики політик, compose та make_multiplier, інжекцію годинника й текст звіту:

```
57     assert "total" not in order
58     assert new is not order
59
60
61 def test_callable_policies(sample_orders: list[Order]) -> None:
62     process = make_processor(
63         accept=lambda s: s >= 50,
64         apply_discount=lambda s: s * 0.9,
65         apply_tax=lambda s: s * 1.2,
66     )
67     result = process(sample_orders)
68     assert [o["id"] for o in result["orders"]] == [1, 4]
69     assert result["revenue"] == pytest.approx((150 + 111) * 0.9 * 1.2)
70
71
72 def test_policy_factories() -> None:
73     assert min_total_filter(100)(99.9) is False
74     assert min_total_filter(100)(100) is True
75     assert percent_discount(0.25)(200) == 150.0
76     assert flat_tax(0.2)(100) == pytest.approx(120.0)
77
78
79 def test_compose_and_multiplier() -> None:
80     discount = compose(flat_tax(0.2), percent_discount(0.5))
81     assert discount(100) == pytest.approx(60.0)
82     assert make_multiplier(3)(7) == 21
83     assert compose(make_multiplier(2), make_multiplier(5))(1) == 10
84
85
86 def test_stamp_total_uses_injected_clock() -> None:
87     assert stamp_total(10.0, lambda: 123.0) == (10.0, 123.0)
88
89
90 def test_render_report(sample_orders: list[Order]) -> None:
91     text = render_report(process_orders_pure(sample_orders, **ARGS))
92     assert text.splitlines() == [
93         "Processed: 1 total: 162.00",
94         "Processed: 4 total: 119.88",
95         "Count: 2",
96         "Revenue: 281.88",
97     ]
```

8. Запустив оболонку на вбудованому прикладі: з чотирьох замовлень відібрано два оплачених із сумою не менше 100, до них застосовано знижку 10 % і податок 20 %:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python app.py
Processed: 1 total: 162.00
Processed: 4 total: 119.88
Count: 2
Revenue: 281.88
Stamped: 281.88 at 17:31:53
```

9. Перевірів проєкт автоматичними інструментами: усі 11 тестів pytest проходять, туру --strict не знаходить помилок типів, ruff та black не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: platformdirs-4.12.3
collecting ... collected 11 items

tests\test_core.py::test_referential_transparency PASSED [ 9%]
tests\test_core.py::test_no_mutation PASSED [ 18%]
tests\test_core.py::test_matches_imperative PASSED [ 27%]
tests\test_core.py::test_filters_unpaid_and_small PASSED [ 36%]
tests\test_core.py::test_order_subtotal PASSED [ 45%]
tests\test_core.py::test_with_total_returns_new_dict PASSED [ 54%]
tests\test_core.py::test_callable_policies PASSED [ 63%]
tests\test_core.py::test_policy_factories PASSED [ 72%]
tests\test_core.py::test_compose_and_multiplier PASSED [ 81%]
tests\test_core.py::test_stamp_total_uses_injected_clock PASSED [ 90%]
tests\test_core.py::test_render_report PASSED [100%]

===== 11 passed in 0.03s =====

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python -m mypy .
Success: no issues found in 5 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab01> python -m black --check .
All done! 🌟 📦 🌟
5 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я розібрав поняття чистої функції, референтної прозорості та побічних ефектів і переписав імперативну обробку замовлень у функціональному стилі: прибрав глобальний стан, мутації вхідних даних і `print` із ядра, виніс I/O в окрему оболонку, а політики знижки, податку та фільтрації передав через `typing.Callable` у фабрику `make_processor`. Тести на референтну прозорість і відсутність мутацій, а також перевірки туру, `ruff` і `black` підтвердили коректність і чистоту реалізації.